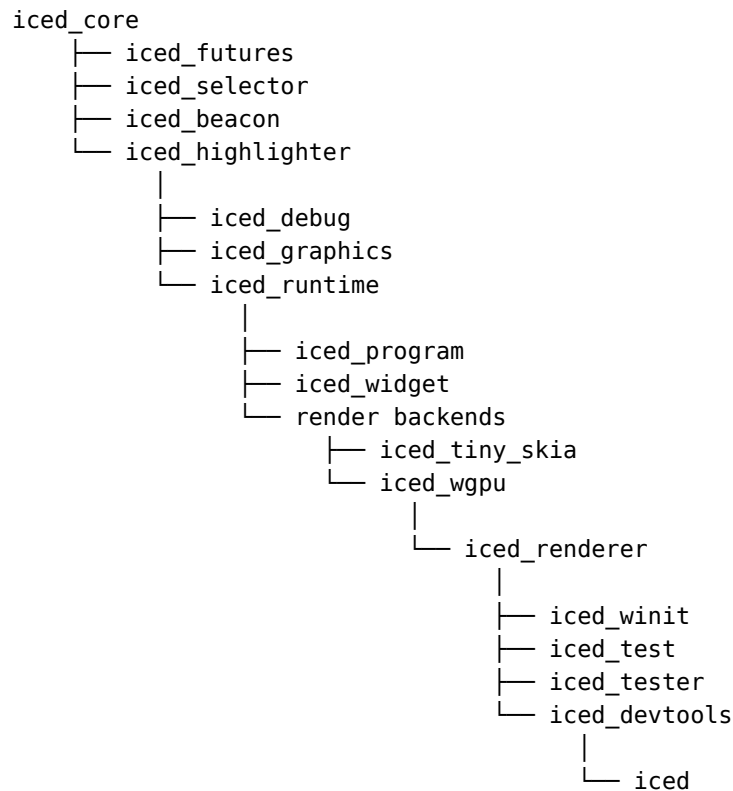


iced: overview

plan

Yes. For a genuinely bottom-up analysis, follow the internal crate graph rather than starting with the public iced API.

A good attack order is:



Some branches overlap, but this captures the useful conceptual progression.

Recommended study phases

1. Pure foundations: iced_core

Start at `[iced/core/src]` (`/home/tomatozoo/projects/seemath/analyze/iced/iced/core/src`).

This is the most important layer. It defines the vocabulary used by everything above it without committing to a window system or rendering backend.

Suggested order inside core:

1. Geometry and styling primitives

- Point, Size, Rectangle, Vector
- Color, Background, Border, Gradient
- Length, Padding, Alignment

2. Input and events

- mouse cursor and events
- keyboard keys and modifiers
- Event and `event::Status`

3. Layout

- `[layout/limits.rs](/home/tomatozoo/projects/seemath/analyze/iced/iced/core/src/layout/limits.rs)`
- `[layout/node.rs](/home/tomatozoo/projects/seemath/analyze/iced/iced/core/src/layout/node.rs)`
- `[layout/flex.rs](/home/tomatozoo/projects/seemath/analyze/iced/iced/core/src/layout/flex.rs)`

4. Widget abstraction

- `Widget`
- `Element`
- `Tree`
- widget state and tags
- `Shell`
- `Overlay`

Good tests:

- `Rectangle::intersection` and containment
- layout limit resolution for `Fill`, `Shrink`, and fixed lengths
- flex distribution with mixed child lengths
- layout node translation
- cursor hit testing
- event-status propagation
- widget tree reconciliation after child types change

The widget tree tests are especially valuable because they explain why Iced widgets can preserve local state despite rebuilding the UI description.

2. Effects and asynchronous execution: `iced_futures`

Study `[iced/futures/src](/home/tomatozoo/projects/seemath/analyze/iced/iced/futures/src)`.

Focus on:

- `Executor`
- `Runtime`
- `Subscription`
- subscription recipes and identity
- subscription tracking
- event streams

Good tests:

- equal subscriptions have stable identity
- changing subscription data restarts the stream
- removed subscriptions are stopped
- mapped subscription output becomes application messages
- commands/tasks complete and emit messages
- batched effects all execute

This establishes the difference between persistent subscriptions and one-shot tasks.

3. Small supporting crates

These can be covered quickly:

- [iced/selector](/home/tomatozoo/projects/seemath/analyze/iced/iced/selector): locating widgets through operations
- [iced/beacon](/home/tomatozoo/projects/seemath/analyze/iced/iced/beacon): instrumentation/communication support
- [iced/highlighter](/home/tomatozoo/projects/seemath/analyze/iced/iced/highlighter): syntax highlighting
- [iced/debug](/home/tomatozoo/projects/seemath/analyze/iced/iced/debug): instrumentation and debug state

For understanding ordinary application behavior, selector matters more than beacon or highlighter.

4. Renderer-independent graphics: iced_graphics

Study [iced/graphics/src](/home/tomatozoo/projects/seemath/analyze/iced/iced/graphics/src).

Important concepts:

- renderer/compositor separation
- primitives
- layers
- viewport
- damage tracking
- geometry paths
- geometry cache
- text paragraph/editor abstractions
- image storage/cache

Good tests:

- viewport logical-to-physical conversion
- damage-region merging
- geometry cache invalidation
- layer ordering
- clipping behavior
- text paragraph bounds
- image cache insertion and eviction

Avoid digging into wgpu immediately. First understand what the higher layers ask a renderer to do.

5. Runtime and UI traversal: iced_runtime

Study [iced/runtime/src](/home/tomatozoo/projects/seemath/analyze/iced/iced/runtime/src), particularly [user_interface.rs](/home/tomatozoo/projects/seemath/analyze/iced/iced/runtime/src/user_interface.rs).

This is where the core pieces become a functioning UI:

Element

- widget tree reconciliation
- layout
- event dispatch
- messages/effects
- draw

Good tests:

- building a UserInterface

- relay layout after bounds change
- event traversal through nested widgets
- captured versus ignored events
- message publication through Shell
- redraw requests
- clipboard operations
- widget operations and selectors
- overlay event priority

This phase will probably give you the largest jump in understanding.

6. Individual widgets: `iced_widget`

Study `[iced/widget/src](/home/tomatozoo/projects/seemath/analyze/iced/iced/widget/src)`.

Do not read every widget sequentially. Use increasing complexity:

1. `space`
2. `text`
3. `container`
4. `row` and `column`
5. `button`
6. `checkbox`
7. `slider`
8. `scrollable`
9. `text_input`
10. `pick_list` and overlays
11. `pane_grid`
12. `canvas`

For each widget, use the same test template:

- What persistent state does it store?
- How does it calculate layout?
- Which events does it capture?
- When does it publish a message?
- What mouse interaction does it report?
- What primitive does it draw?
- Does it create an overlay?

Test widgets through Iced's UI-testing facilities where possible instead of asserting renderer implementation details.

7. Program architecture: `iced_program`

Study `[iced/program/src](/home/tomatozoo/projects/seemath/analyze/iced/iced/program/src)`.

Focus on how application state is connected to:

- `view`
- `update`
- `subscriptions`
- `tasks`
- `themes`
- `window configuration`

Good tests:

- message → state transition
- update returning a task
- task result → subsequent message
- subscription changes based on model state
- rebuilding the view after state changes

At this point, the Elm-style architecture should feel like a thin layer over the runtime you already studied.

8. Rendering backends

Study the software renderer first:

- [iced/tiny_skia](/home/tomatozoo/projects/seemath/analyze/iced/iced/tiny_skia)
- then [iced/wgpu](/home/tomatozoo/projects/seemath/analyze/iced/iced/wgpu)
- finally [iced/renderer](/home/tomatozoo/projects/seemath/analyze/iced/iced/renderer)

tiny_skia is easier to reason about because it has fewer GPU concepts. Use it to understand how graphics primitives become pixels.

Suggested tests:

- render a colored quad and inspect pixels
- clipping
- opacity composition
- layer ordering
- text or geometry rendering
- identical scene producing identical output

Then study wgpu concepts:

- primitive preparation
- pipelines
- buffers
- texture/image atlases
- text rendering
- compositor and surface presentation

Most GPU tests will be integration tests and may be hardware-dependent, so keep them later.

9. Window/event-loop integration: iced_winit

Study [iced/winit/src](/home/tomatozoo/projects/seemath/analyze/iced/iced/winit/src).

Focus on:

- conversion from winit events to Iced events
- window state
- redraw scheduling
- event-loop proxy
- surface lifecycle
- clipboard integration

Good tests:

- key and mouse event conversion
- modifiers conversion
- scale-factor handling

- cursor-position conversion
- redraw-decision logic

Avoid using real windows for most tests. Test conversion and state-transition functions directly where possible.

10. Testing infrastructure and public facade

Finish with:

- [iced/test](/home/tomatozoo/projects/seemath/analyze/iced/iced/test)
- [iced/tester](/home/tomatozoo/projects/seemath/analyze/iced/iced/tester)
- [iced/devtools](/home/tomatozoo/projects/seemath/analyze/iced/iced/devtools)
- [iced/src](/home/tomatozoo/projects/seemath/analyze/iced/iced/src)

Finally study a few examples:

1. counter
2. todos
3. stopwatch
4. pane_grid
5. custom_widget
6. custom_quad

Now you can trace each example all the way from public API calls down to event handling, layout, and rendering.

Structure the analysis crate by layers

I would organize your tests approximately like this:

```
analysis/
├─ Cargo.toml
├─ tests/
│   ├── core_geometry.rs
│   ├── core_layout.rs
│   ├── core_widget_tree.rs
│   ├── futures_subscription.rs
│   ├── graphics_primitives.rs
│   ├── runtime_user_interface.rs
│   ├── widget_button.rs
│   ├── widget_scrollable.rs
│   ├── widget_text_input.rs
│   ├── program_lifecycle.rs
│   ├── renderer_pixels.rs
│   └─ winit_conversion.rs
```

Your current [analysis/Cargo.toml](/home/tomatozoo/projects/seemath/analyze/iced/analysis/Cargo.toml) only depends on the top-level iced crate. For bottom-up analysis, add direct path dependencies on the internal crates as you reach them. That makes each test's architectural boundary explicit:

```
[dependencies]
iced_core = { path = "../iced/core", features = ["advanced"] }
iced_futures = { path = "../iced/futures" }
iced_graphics = { path = "../iced/graphics" }
iced_runtime = { path = "../iced/runtime" }
iced_widget = { path = "../iced/widget" }
iced_program = { path = "../iced/program" }
```

Add renderers and window integration only in their respective phases. Keeping default GPU/window features out of the early tests will make them faster, more deterministic, and easier to understand.

The highest-value route is:

- core layout
- widget tree
- subscriptions/tasks
- UserInterface traversal
- simple widgets
- Program
- software rendering
- winit
- wgpu

That order follows dependencies while also preserving the conceptual story of how Iced works.